

Generic Stack<T> Collection Class in C# with Examples

In this article, I am going to discuss **Generic Stack<T> Collection Class in C#** with Examples. Please read our previous article where we discussed the difference between [List vs Dictionary in C#](#) with Examples. The Stack<T> is a Generic collection that stores elements in LIFO style (Last In First Out). C# includes both the Generic Stack<T> and **Non-Generic Stack** collection classes. It is recommended by Microsoft to use the Generic Stack<T> Collection as it is type-safe and boxing and unboxing is not required. Here in this article, we will discuss the Generic Stack Collection Class in C# with Examples. At the end of this article, you will understand the following pointers.

1. [What is Generic Stack<T> in C#?](#)
2. [How to Create a Generic Stack<T> Collection in C#?](#)
3. [How to Add Elements into a Stack<T> Collection in C#?](#)
4. [How to access a Generic Stack<T> Collection in C#?](#)
5. [How to Remove Elements from a Generic Stack<T> Collection in C#?](#)
6. [How to get the Topmost Element of a Generic Stack in C#?](#)
7. [What is the difference between the Pop\(\) and Peek\(\) methods?](#)
8. [How to check whether an element exists or not in the Generic Stack<T> Collection in C#?](#)
9. [How to copy a Generic Stack to an Existing Array in C#?](#)
10. [Generic Stack Collection Class in C# with Complex Type](#)
11. [Generic Stack vs Non-Generic Stack in C#](#)

What is Generic Stack<T> in C#?

The Generic Stack in C# is a collection class that works on the principle of Last In First Out (LIFO) and this class is present in System.Collections.Generic namespace. The Generic Stack Collection is used when we need Last In First Out (LIFO) access to items.

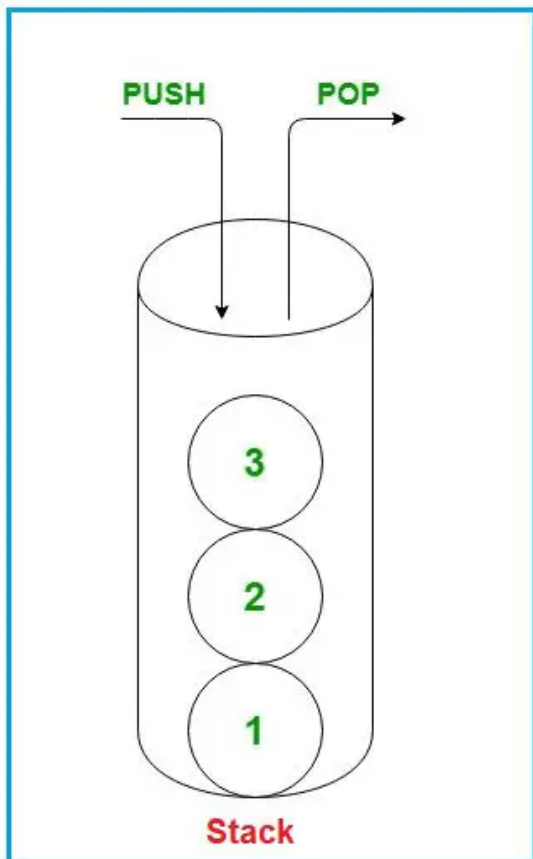
This Stack collection class is analogous to a stack of books. For example, if we want to add a new book to the stack of books, then we will only place it on top of all the already existing books. Similarly, if we want to remove a book from the stack, then we will only remove the one that we have last added. For a better understanding, please have a look at the following image.

Last Inserted Item will be Taken First



Stack - LIFO (Last in First Out)

The stack collection class also operates in a similar fashion. The last item to be added (pushed) to the stack will be the first item to be removed (popped) from the stack. When we add an element into the stack, it is called pushing the element and when we remove an element from the stack, it is called popping the element. For a better understanding, please have a look at the below image.



The capacity of a Stack is the number of elements the Stack can hold. As elements are added to a Stack, the capacity is automatically increased. In Generic Stack Collection, we can store duplicate elements. A Stack can also accept null as a valid value for reference types.

Methods, Properties, and Constructors of Generic Stack<T> Collection Class in C#:

If you go to the definition of Generic Stack<T> Collection class, then you will see the following. As you can see the Generic Stack class implements the IEnumerable<T>, IEnumerable, IReadOnlyCollection<T>, and ICollection interfaces.

```
namespace System.Collections.Generic
{
    ... public class Stack<T> : IEnumerable<T>, IEnumerable, IReadOnlyCollection<T>, ICollection
    {
        ... public Stack();
        ... public Stack(IEnumerable<T> collection);
        ... public Stack(int capacity);

        ... public int Count { get; }

        ... public void Clear();
        ... public bool Contains(T item);
        ... public void CopyTo(T[] array, int arrayIndex);
        ... public Stack<T>.Enumerator GetEnumerator();
        ... public T Peek();
        ... public T Pop();
        ... public void Push(T item);
        ... public T[] ToArray();
        ... public void TrimExcess();
        ... public bool TryPeek([MaybeNullWhen(false)] out T result);
        ... public bool TryPop([MaybeNullWhen(false)] out T result);

        ... public struct Enumerator : IEnumerator<T>, IEnumerator, IDisposable
        {
            ... public T Current { get; }

            ... public void Dispose();
            ... public bool MoveNext();
        }
    }
}
```

How to Create a Generic Stack<T> Collection in C#?

The Generic Collection Stack<T> class in C# provides the following three constructors to create an instance of the Generic Stack<T> class.

1. **Stack():** It is used to initialize a new instance of the Generic Stack class that is empty and has the default initial capacity.
2. **Stack(IEnumerable<T> collection):** It is used to initialize a new instance of the Generic Stack class that contains elements copied from the specified collection and has sufficient capacity to accommodate the number of elements copied. Here, the parameters collection specifies the collection to copy elements from. If the collection is null, then it will throw ArgumentNullException.

3. **Stack(int capacity):** It is used to initialize a new instance of the Generic Stack class that is empty and has the specified initial capacity or the default initial capacity, whichever is greater. Here, the parameter capacity specifies the initial number of elements that the Stack can contain. If capacity is less than zero, then it will throw `ArgumentOutOfRangeException`.

Let's see how to create an instance of Generic Stack using the `Stack()` constructor:

Step1:

As the `Generic Stack<T>` class belongs to `System.Collections.Generic` namespace, so first, we need to include `System.Collections.Generic` namespace into our program as follows:

```
using System.Collections.Generic;
```

Step2:

Next, we need to create an instance of the Generic Stack class using the `Stack()` constructor as follows:

```
Stack<type> stack = new Stack<type>();
```

Here, the type can be any built-in data type like `int`, `double`, or `string`, or any user-defined data type like `Customer`, `Employee`, `Product`, etc.

How to Add Elements into a Stack<T> Collection in C#?

If you want to add elements to a generic stack collection in C#, then you need to use the following `Push()` method of the `Stack<T>` class.

1. **Push(T item):** The `Push(T item)` method is used to insert an element on top of the Stack. Here, the parameter `item` specifies the element to push onto the Stack. The value can be null for a reference type i.e. when `T` is a reference type, then we can push null to the stack.

For example,

```
Stack<int> stack = new Stack<int>();
```

The above statement will create a generic stack of integer types. So, here we can only push integer-type elements on the stack. If we try to push anything other than an integer then we will get a compile-time error.

```
stack.Push(10);
```

```
stack.Push(20);
```

```
stack.Push("Hello"); //Compile-Time Error
```

Note: We cannot add elements into a Stack using Collection Initializer and the reason for this is the `Generic Stack<T>` Collection Class does not have an `Add` method. So, the point that you need to remember is if any collection class does not have the `Add` method then you cannot use Collection Initializer to add elements. If you try then you will get the compile-time error as shown in the below image.

```
Stack<int> stack = new Stack<int>()
{
    10, 20
};
```

 `struct System.Int32`

Represents a 32-bit signed integer.

'Stack<int>' does not contain a definition for 'Add' and no accessible extension method 'Add' accepting a first argument of type 'Stack<int>'.

How to access a Generic Stack<T> Collection in C#?

We can access all the elements of the Generic Stack<T> collection in C# using a for each loop as follows.

```
foreach (var item in stack)
{
    Console.WriteLine(item);
}
```

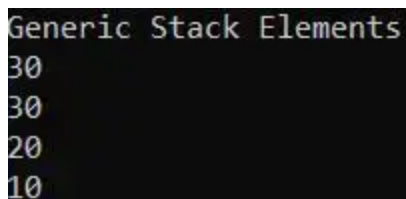
Note: You cannot use a for loop to access the elements of a generic stack collection and the reason for this is the Generic Stack<T> collection class does not have any integer indexer.

Example to Understand How to Create a Generic Stack and Add Elements in C#:

For a better understanding of how to create a Generic Stack, how to add elements to a stack, and how to access all the elements from a stack in C#, please have a look at the following example.

```
using System;
using System.Collections.Generic;
namespace GenericStackCollection
{
    public class Program
    {
        public static void Main()
        {
            Stack<int> stack = new Stack<int>();
            stack.Push(10);
            stack.Push(20);
            stack.Push(30);
            //Adding Duplicate
            stack.Push(30);
            //As int is not a Reference type so null can not be accepted by the stack
            //stack.Push(null); //Compile-Time Error
            //As the stack is integer type, so string values can not be accepted
            //stack.Push("Hello"); //Compile-Time Error
            Console.WriteLine("Generic Stack Elements");
            foreach (var item in stack)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:



```
Generic Stack Elements
30
30
20
10
```

How to Remove Elements from a Generic Stack<T> Collection in C#?

In Stack, we are only allowed to remove elements from the top of the stack. The Generic Stack Collection Class in C# provides the following two methods to remove elements.

1. **Pop():** This method is used to remove and return the object at the top of the Generic Stack. It returns the Object (element) removed from the top of the Stack.
2. **Clear():** This method is used to remove all objects from the Generic Stack.

Let us see an example to understand the Pop and Clear methods of Generic Stack<T> Collection Class in C#. Please have a look at the following example which shows the use of the Pop and Clear method.

```
using System;
using System.Collections.Generic;
namespace GenericStackCollection
{
    public class Program
    {
        public static void Main()
        {
            //Creating a Generic Stack to Store Integer Elements
            Stack<int> genericStack = new Stack<int>();
            //Pushing Elements to the Stack using Push Method
            genericStack.Push(10);
            genericStack.Push(20);
            genericStack.Push(30);
            genericStack.Push(50);
            genericStack.Push(25);
            //Printing the Stack Elements using Foreach loop
            Console.WriteLine($"All Generic Stack Elements, Count:
{genericStack.Count}");
            foreach (var element in genericStack)
            {
                Console.WriteLine(element);
            }
            // Removing and Returning an Element from the Generic Stack using
            Console.WriteLine($"\\nDeleted Element: {genericStack.Pop()}");

            //Printing Elements After Removing the Last Added Element
            Console.WriteLine($"\\nAll Stack Elements After Deletion: Count
{genericStack.Count}");
            foreach (var element in genericStack)
            {
                Console.WriteLine($"{element} ");
            }
            //Removing All Elements from Generic Stack using Clear Method
            genericStack.Clear();
            Console.WriteLine($"\\nAll Stack Elements Counts After Clear: Count
{genericStack.Count}");
        }
    }
}
```

```

        Console.ReadKey();
    }
}

```

Output:

```

All Generic Stack Elements, Count: 5
25
50
30
20
10

Deleted Element: 25

All Stack Elements After Deletion: Count 4
50
30
20
10

All Stack Elements Counts After Clear: Count 0

```

How to get the Topmost Element of a Generic Stack in C#?

The Generic Stack class in C# provides the following two methods to get the topmost element of the Stack.

1. **Pop():** This method is used to remove and return the object at the top of the Generic Stack. It returns the Object (element) removed from the top of the Stack. If there is no object (or element) present in the stack and if you are trying to remove an item or object from the stack using the pop() method then it will throw an exception i.e. System.InvalidOperationException
2. **Peek():** This method is used to return the object at the top of the Generic Stack without removing it. If there is no object (or element) present in the stack and if you are trying to return an item (object) from the stack using the peek() method then it will throw an exception i.e. System.InvalidOperationException.

For a better understanding, please have a look at the below example which shows how to get the topmost element from the stack.

```

using System;
using System.Collections.Generic;
namespace GenericStackCollection
{
    public class Program
    {
        public static void Main()
        {
            //Creating a Generic Stack to Store Integer Elements

```

```

Stack<int> genericStack = new Stack<int>();
//Pushing Elements to the Stack using Push Method
genericStack.Push(10);
genericStack.Push(20);
genericStack.Push(30);
genericStack.Push(50);
genericStack.Push(25);
//Printing the Stack Elements using Foreach loop
Console.WriteLine($"All Generic Stack Elements, Count:
{genericStack.Count}");
    foreach (var element in genericStack)
    {
        Console.WriteLine(element);
    }
// Removing and Returning an Element from the Generic Stack using
Console.WriteLine($"\\nPop Element: {genericStack.Pop()}");

//Printing Elements After Removing the Last Added Element
Console.WriteLine($"All Stack Elements After Pop: Count
{genericStack.Count}");
    foreach (var element in genericStack)
    {
        Console.WriteLine($"{element} ");
    }
// Returning an Element from the Generic Stack using Peek method v
Removing
Console.WriteLine($"\\nPeek Element: {genericStack.Peek()}");
//Printing Elements After Peek the Last Added Element
Console.WriteLine($"All Stack Elements After Peek: Count
{genericStack.Count}");
    foreach (var element in genericStack)
    {
        Console.WriteLine($"{element} ");
    }
    Console.ReadKey();
}
}
}

```

Output:

```
All Generic Stack Elements, Count: 5
25
50
30
20
10

Pop Element: 25
All Stack Elements After Pop: Count 4
50
30
20
10

Peek Element: 50
All Stack Elements After Peek: Count 4
50
30
20
10
```

What is the difference between the Pop() and Peek() methods?

The Pop() method removes and returns the item at the top of the stack, whereas the Peek() method returns the item from the top of the stack, without removing it. This is the only difference between these two methods of Stack class in C#.

How to check whether an element exists or not in the Generic Stack<T> Collection in C#?

If you want to check whether an element exists or not in the Generic Stack<T> Collection, then you need to use the following Contains() method provided by the Generic Stack Class in C#. Even, you can also use this method to search for an element in the given stack.

1. **Contains(T item):** The Contains(T item) method is used to determine whether an element exists in the generic stack or not. It returns true if the item is found in the generic Stack; otherwise, false. Here, the parameter item specifies the element to locate in the Stack. The value can be null for a reference type.

Let us understand Contains(T item) Method with an example. The following example shows how to use the Contains() method of the Generic Stack Collection class in C#.

```
using System;
using System.Collections.Generic;
namespace GenericStackCollection
{
    public class Program
    {
```

```

public static void Main()
{
    //Creating a Generic Stack to Store Integer Elements
    Stack<int> genericStack = new Stack<int>();
    //Pushing Elements to the Stack using Push Method
    genericStack.Push(10);
    genericStack.Push(20);
    genericStack.Push(30);
    genericStack.Push(50);
    genericStack.Push(25);
    //Printing the Stack Elements using Foreach loop
    Console.WriteLine($"All Generic Stack Elements, Count:
{genericStack.Count}");
    foreach (var element in genericStack)
    {
        Console.WriteLine(element);
    }
    Console.WriteLine($"\\nIs Value 50 Present in the Generic Stack:
{genericStack.Contains(50)}");
    Console.WriteLine($"\\nIs Value 90 Present in the Generic Stack:
{genericStack.Contains(90)}");
    Console.ReadKey();
}
}

```

Output:

```

All Generic Stack Elements, Count: 5
25
50
30
20
10

Is Value 50 Present in the Generic Stack: True

Is Value 90 Present in the Generic Stack: False

```

Note: The Contains(T item) method of the Generic Stack Class takes O(n) time to check if the element exists in the stack. This should be taken into consideration while using this method.

How to copy a Generic Stack to an Existing Array in C#?

In order to copy a Generic Stack to an Existing Array in C#, we need to use the following CopyTo method of the Generic Stack Collection Class.

1. **CopyTo(T[] array, int arrayIndex):** This method is used to copy the Stack elements to an existing one-dimensional Array, starting at the specified array index. Here, the parameter array specifies the

one-dimensional array that is the destination of the elements copied from the generic stack. The Array must have zero-based indexing. The arrayIndex parameter specifies the zero-based index in the array at which copying begins. If the parameter array is null, then it will throw ArgumentNullException. If the parameter index is less than zero, then it will throw ArgumentOutOfRangeException. If the number of elements in the source Generic Stack is greater than the available space from arrayIndex to the end of the destination array, then it will throw ArgumentException.

This method works on one-dimensional arrays and does not change the state of the generic stack. The elements are ordered in the array in the same way as the order of the elements from the start of the stack to the end. Let us see an example for a better understanding of the CopyTo(T[] array, int arrayIndex) method of the Generic Stack<T> Collection Class in C#.

```
using System;
using System.Collections.Generic;
namespace GenericStackCollection
{
    public class Program
    {
        public static void Main()
        {
            //Creating a Generic Stack to Store Integer Elements
            Stack<int> genericStack = new Stack<int>();
            //Pushing Elements to the Stack using Push Method
            genericStack.Push(10);
            genericStack.Push(20);
            genericStack.Push(30);
            genericStack.Push(50);
            genericStack.Push(25);
            //Printing the Stack Elements using Foreach loop
            Console.WriteLine($"All Generic Stack Elements, Count:
{genericStack.Count}");
            foreach (var element in genericStack)
            {
                Console.WriteLine(element);
            }
            //Copying the stack to an object array
            int[] stackCopy = new int[5];
            genericStack.CopyTo(stackCopy, 0);
            Console.WriteLine("\nStack Copy Array Elements:");
            foreach (var item in stackCopy)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
All Generic Stack Elements, Count: 5
25
50
30
20
10

Stack Copy Array Elements:
25
50
30
20
10
```

Generic Stack Collection Class in C# with Complex Type.

As of now, we have used the Generic Stack<T> Collection class with primitive data types such as int. Now, let us proceed further and see how to use the Generic Stack<T> Collection class in C# with complex types such as Employee, Customer, Product, etc. For a better understanding, please have a look at the below example where we use the Generic Stack Collection with the user-defined Employee and perform different kinds of Operations. The following code is self-explained, so please go through the comment lines.

```
namespace GenericStackDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            //Create Employee object
            Employee emp1 = new Employee()
            {
                ID = 101,
                Name = "Pranaya",
                Gender = "Male",
                Salary = 20000
            };
            Employee emp2 = new Employee()
            {
                ID = 102,
                Name = "Priyanka",
                Gender = "Female",
                Salary = 30000
            };
            Employee emp3 = new Employee()
            {
                ID = 103,
                Name = "Anurag",
                Gender = "Male",
                Salary = 40000
            }
        }
    }
}
```

```

};
Employee emp4 = new Employee()
{
    ID = 104,
    Name = "Sambit",
    Gender = "Female",
    Salary = 40000
};
Employee emp5 = new Employee()
{
    ID = 105,
    Name = "Preety",
    Gender = "Female",
    Salary = 50000
};
// Create a Generic Stack of Employees
Stack<Employee> stackEmployees = new Stack<Employee>();
// To add an item into the stack, use the Push() method.
// emp1 is inserted at the top of the stack
stackEmployees.Push(emp1);
// emp2 will be inserted on top of emp1 and now is on top of the stack
stackEmployees.Push(emp2);
// emp3 will be inserted on top of emp2 and now is on top of the stack
stackEmployees.Push(emp3);
// emp4 will be inserted on top of emp3 and now is on top of the stack
stackEmployees.Push(emp4);
// emp5 will be inserted on top of emp4 and now is on top of the stack
stackEmployees.Push(emp5);
// If you need to loop thru each items in the stack, then we can use a
foreach loop
// in the same way as we use it with other collection classes.
// The foreach loop will only iterate thru the items in the stack,
not remove them.
// Notice that the items from the stack are retrieved in LIFO (Last In, First Out), order.
// The last element added to the Stack is the first one to be removed
Console.WriteLine("Retrive Using Foreach Loop");
foreach (Employee emp in stackEmployees)
{
    Console.WriteLine(emp.ID + " - " + emp.Name + " - " + emp.Gender + " - " + emp.Salary);
    Console.WriteLine("Items left in the Stack = " + stackEmployees.Count);
}
Console.WriteLine("-----");
// To retrieve an item from the stack, use the Pop() method.
// This method removes and returns an object at the top of the stack
// Since emp5 object is the one that is pushed onto the stack last, the first
object will be
// first to be removed and returned from the stack by the Pop() method
Console.WriteLine("Retrive Using Pop Method");
Employee e1 = stackEmployees.Pop();

```

```

        Console.WriteLine(e1.ID + " - " + e1.Name + " - " + e1.Gender + "
e1.Salary));
        Console.WriteLine("Items left in the Stack = " + stackEmployees.Count);
        Employee e2 = stackEmployees.Pop();
        Console.WriteLine(e2.ID + " - " + e2.Name + " - " + e2.Gender + "
e2.Salary));
        Console.WriteLine("Items left in the Stack = " + stackEmployees.Count);
        Employee e3 = stackEmployees.Pop();
        Console.WriteLine(e3.ID + " - " + e3.Name + " - " + e3.Gender + "
e3.Salary));
        Console.WriteLine("Items left in the Stack = " + stackEmployees.Count);
        Employee e4 = stackEmployees.Pop();
        Console.WriteLine(e4.ID + " - " + e4.Name + " - " + e4.Gender + "
e4.Salary));
        Console.WriteLine("Items left in the Stack = " + stackEmployees.Count);
        Employee e5 = stackEmployees.Pop();
        Console.WriteLine(e5.ID + " - " + e5.Name + " - " + e5.Gender + "
e5.Salary));
        Console.WriteLine("Items left in the Stack = " + stackEmployees.Count);
        Console.WriteLine("-----");
        // Now there will be no items left in the stack.
        // So, let's push the five objects once again
        stackEmployees.Push(emp1);
        stackEmployees.Push(emp2);
        stackEmployees.Push(emp3);
        stackEmployees.Push(emp4);
        stackEmployees.Push(emp5);
        // To retrieve an item that is present at the top of the stack,
        // without removing it, then use the Peek() method.
        Console.WriteLine("Retrive Using Peek Method");
        Employee e105 = stackEmployees.Peek();
        Console.WriteLine(e105.ID + " - " + e105.Name + " - " + e105.Gender + "
e105.Salary));
        Console.WriteLine("Items left in the Stack = " + stackEmployees.Count);
        Employee e104 = stackEmployees.Peek();
        Console.WriteLine(e104.ID + " - " + e104.Name + " - " + e104.Gender + "
e104.Salary));
        Console.WriteLine("Items left in the Stack = " + stackEmployees.Count);
        Console.WriteLine("-----");
        // To check if an item exists in the stack, use Contains() method.
        if (stackEmployees.Contains(emp3))
        {
            Console.WriteLine("Emp3 is in stack");
        }
        else
        {
            Console.WriteLine("Emp3 is not in stack");
        }
        Console.ReadKey();
    }
}

```

```

    }
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
        public int Salary { get; set; }
    }
}

```

Output:

```

Items left in the Stack = 5
104 - Sambit - Female - 40000
Items left in the Stack = 5
103 - Anurag - Male - 40000
Items left in the Stack = 5
102 - Priyanka - Female - 30000
Items left in the Stack = 5
101 - Pranaya - Male - 20000
Items left in the Stack = 5
-----
Retrive Using Pop Method
105 - Preety - Female - 50000
Items left in the Stack = 4
104 - Sambit - Female - 40000
Items left in the Stack = 3
103 - Anurag - Male - 40000
Items left in the Stack = 2
102 - Priyanka - Female - 30000
Items left in the Stack = 1
101 - Pranaya - Male - 20000
Items left in the Stack = 0
-----
Retrive Using Peek Method
105 - Preety - Female - 50000
Items left in the Stack = 5
105 - Preety - Female - 50000
Items left in the Stack = 5
-----
Emp3 is in stack

```

Generic Stack vs Non-Generic Stack in C#

1. The Generic Stack<T> Collection Class is defined under System.Collections.Generic namespace where the Non-Generic Stack Collection Class is defined under System.Collections namespace.
2. The Generic Stack<T> Class in C# can only store the same type of elements whereas the Non-Generic Stack Class can store the same or different types of elements as it operates on the object

data type.

3. In Generic Stack<T>, we need to define the type of elements that we want to store in the stack. On the other hand, in a Non-Generic Stack, there is no need to define the type of elements that we want to store in the stack because it operates on object data type i.e. we can store any type of data.
4. The Generic Stack<T> is type-safe whereas the Non-Generic Stack is not type-safe. Boxing and Unboxing are not required in Generic Stack whereas Boxing and Unboxing are required in the Non-Generic Stack which will degrade the application performance.
5. Because of type mismatch, we may get runtime exceptions in a non-generic stack whereas in the case of the generic stack, if we are storing other types of elements, then we will get compile time error.

C# Generic Stack<T> Collection Class Summary:

The following are some important points that you need to remember while working with Generic Stack Collection Class in C#.

1. The Stack Collection is used to store a collection of the same type of elements in a LIFO (Last in, First out) style, i.e., the element which added last will come out first.
2. As Stack<T> is a Generic Collection, so it comes under System.Collection.Generic namespace.
3. The Generic Stack<T> Collection stores elements of the specified type. It provides compile-time type checking and doesn't perform boxing-unboxing because it is generic.
4. By using the Push() method, we can add elements to a stack collection. Here, we cannot use collection-initializer syntax to add elements to a stack.
5. The Pop() method will remove and return the topmost element from the stack. It does not support an indexer.
6. The Peek() method will return the last (top-most) inserted element of the stack, and it won't delete the element from the stack.
7. Stack Collection is very useful to store temporary data in the Last In First Out (LIFO) style, where you might want to delete an element after retrieving its value.
8. New elements are always added at the end of the Stack<T>.
9. Elements are removed from the end of the Stack<T>
10. Duplicate elements are allowed to be stored in a Stack.
11. As a Stack<T> maintains a LIFO collection of objects, so you may use Stack<T> when you need to access the information in reverse order.
12. To iterate over the elements of Stack<T>, we could use the for each loop.

In the next article, I am going to discuss [Generic Queue Collection Class in C#](#) with Examples. Here, in this article, I try to explain **Generic Stack Collection Class in C#** with Examples. I hope this Generic Stack<T> Collection Class in C# article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article.